

Amérique du Nord – 2026 – sujet2

Exercice 2 (6 points)

Cet exercice porte sur la programmation Python en général, la programmation orientée objet en particulier et la structure de données d'arbre.

La WTA (Women's Tennis Association) est l'instance dirigeante du tennis professionnel international féminin, responsable de l'organisation du circuit « WTA Tour » et de l'établissement des classements mondiaux des joueuses. Dans cet exercice, on ne considère que des matchs de tennis en simple, c'est-à-dire des matchs qui opposent uniquement deux joueuses.

PARTIE A

On dispose d'une classe `Joueuse` pour modéliser une joueuse de tennis professionnelle du circuit WTA.

```
1 class Joueuse:
2     def __init__(self, nom, prenom, pays, age, point):
3         ''' nom, prenom et pays sont de type str,
4           age et point de type int '''
5         self.nom = nom
6         self.prenom = prenom
7         self.pays = pays
8         self.age = age
9         # nombre de points WTA
10        self.point = point
11        # nombre de victoires
12        self.victoire = 0
13        # nombre de défaites
14        self.defaite = 0
```

1. Instancier l'objet `pegula` de la classe `Joueuse` qui modélise la joueuse Pegula Jessica de nationalité américaine ("USA"), âgée de 29 ans et ayant 6101 points WTA.

On considère les objets déjà instanciés `gauff`, `paloni`, `sabalenka` et `swiatek` de la classe `Joueuse` représentant respectivement les joueuses :

- Gauff Coco, américaine, âgée de 20 ans et ayant 6063 points WTA ;
 - Paloni Marta, espagnole, âgée de 23 ans et ayant 4843 points WTA ; □ Sabalenka Aryna, biélorusse, âgée de 25 ans et ayant 10541 points WTA ; □ Swiatek Iga, polonaise, âgée de 22 ans et ayant 7470 points WTA.
2. Recopier et compléter le code ci-après de la méthode `ajouter_victoire` de la classe `Joueuse` permettant d'ajouter une victoire à la joueuse en question ainsi qu'une défaite à son adversaire objet de la classe `Joueuse`.

```
1 def ajouter_victoire(self, adversaire):
2     ...
3     ...
```

Lors de la finale du dernier tournoi des masters Marta Paloni a été battue par Iga Swiatek.

3. Écrire une instruction utilisant la méthode `ajouter_victoire` pour prendre en compte l'issue de ce match.

On souhaite classer les joueuses à l'aide de leurs points WTA. Pour cela il est possible en Python redéfinir l'opérateur de comparaison `<` (strictement inférieur) pour les objets de la classe `Joueuse` en utilisant les points WTA. Ainsi on obtient par exemple :

```
>>> swiatek < paloni
False
>>> swiatek < sabalenka
True
```

On utilise une liste Python pour stocker des objets de la classe `Joueuse`,

```
liste_joueuses = [swiatek, gauff, paloni, sabalenka, pegula]
```

que l'on souhaite trier dans l'ordre croissant des points WTA. Pour cela on choisit le tri par insertion dont le code est donné ci-dessous :

```
1 def tri_insertion(liste):
2     for i in range(1, len(liste)):
3         j = i
4         while j > 0 and liste[j] < liste[j - 1]:
5             # échange les valeurs liste[j] et liste[j-1]
6             liste[j], liste[j - 1] = liste[j - 1], liste[j]
7             j = j - 1
```

4. Préciser, sans justifier, le coût temporel du tri par insertion d'une liste de n éléments dans le pire des cas.

On détaille les étapes du tri par insertion appliqué à la liste `liste_joueuses` dans le tableau ci-dessous, où chaque ligne correspond à un échange entre deux éléments de la liste.

Étape	Contenu de <code>liste_joueuses</code>
0	[swiatek, gauff, paloni, sabalenka, pegula]
1	[gauff, swiatek, paloni, sabalenka, pegula]
...	

5. Recopier et compléter le tableau ci-dessus en ajoutant autant de lignes que nécessaires.

Un match de tennis se déroule en plusieurs *sets*, et chaque set en plusieurs *jeux*.

Ayant fixé qui est la joueuse 1 et qui est la joueuse 2, on peut décrire l'issue d'un set à l'aide d'un couple d'entier : le premier entier donne le nombre de jeux gagnés par la joueuse 1 lors de ce set, le second le nombre de jeux gagnés par la joueuse 2 lors de ce set. Le score d'un match est alors décrit par une liste de tuples.

Par exemple, en mai 2024, Swiatek affronte Sabalenka en finale du tournoi « WTA

1000 » de Madrid. Ce match s'est déroulé en 3 sets : 7-5, 4-6, 7-6. Swiatek remporte donc la finale 2 sets à

1. La liste de tuples représentant le score de ce match est

```
[(7, 5), (4, 6), (7, 6)].
```

Aucun cas d'égalité n'est possible.

Pour automatiser la gestion des tournois on crée une classe `Match`.

```
1 class Match:
2     def __init__(self, intitule, joueuse1, joueuse2):
3         self.intitule = intitule
4         self.joueuse1 = joueuse1
5         self.joueuse2 = joueuse2
6         self.gagnante = None
7         self.perdante = None
8         self.score = None
```

Cette classe est constituée de :

- `self.intitule`, un intitulé du match sous la forme d'une chaîne de caractères ;
- `self.joueuse1` et `self.joueuse2`, représentant les deux joueuses, deux objets de la classe `Joueuse` ;
- `self.gagnante` et `self.perdante`, représentant la joueuse victorieuse respectivement la joueuse perdante deux objets de la classe `Joueuse` ;
- `self.score`, représentant le score du match sous la forme d'une liste de tuples d'entiers.

Au terme d'un match une fois que le score est connu, on souhaite pouvoir le saisir à l'aide de la méthode `resultat_match(self, score)` de la classe `Match`. Cette méthode prend en paramètre un score sous la forme de liste de tuple de deux entiers et

- enregistre le score du match ;
- détermine la gagnante et la perdante en comptant le nombre de set(s) gagné(s) par chaque joueur par chaque joueuse ;
- ajoute une victoire à la joueuse gagnante et une défaite à la joueuse perdante.

Par exemple, pour créer un objet `finale_mad_24` représentant le match Swiatek vs Sabalenka décrit précédemment, on utilise les deux instructions suivantes.

```
# instantiation de la finale Madrid 2024
finale_mad_24 = Match('finale Madrid 24', swiatek, sabalenka)

# mise-à-jour du score du match
finale_mad_24.resultat_match([(7, 5), (4, 6), (7, 6)])
```

6. Recopier et compléter le code de la méthode `resultat_match` de la classe `Match`. On ajoutera autant de lignes que nécessaire.

```
1 def resultat_match(self, score):
2     self.score = score
3     nb_set_joueuse1 = 0
4     nb_set_joueuse2 = 0
5     # code incomplet
```

Partie B

Afin de pouvoir modéliser chaque tour d'un tournoi de tennis à partir des quarts de finale uniquement, on décide d'utiliser un arbre binaire.

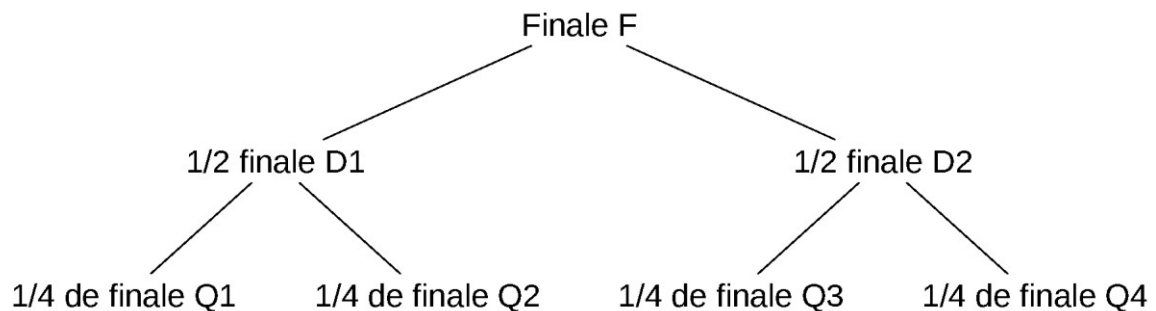


Figure 1. Tournoi de tennis.

7. Justifier qu'un tournoi de tennis peut effectivement être modélisé à l'aide d'un arbre binaire.

On décide d'implémenter un arbre binaire à l'aide de la classe `Arbre` ci-dessous :

```
1 class Arbre:
2     def __init__(self, racine, gauche, droit):
3         self.racine = racine
4         self.gauche = gauche
5         self.droit = droit
```

où `self.racine` est un objet de la classe `Match`, `self.gauche` et `self.droit`, des objets de la classe `Arbre`.

Dans le cas du tournoi de Madrid 2025 on connaît les matchs du tournoi à partir des quarts de finale, où `pilar`, `inie`, `jabeur` sont des objets de la classe `Joueuse` :

```
# quarts de finale
Q1 = Match("Quart de finale 1", gauff, pilar)
Q2 = Match("Quart de finale 2", paloni, inie)
Q3 = Match("Quart de finale 3", pegula, sabalenka)
Q4 = Match("Quart de finale 4", swiatek, jabeur)
# demi-finale
D1 = Match("Demi-finale 1", None, None)
D2 = Match("Demi-finale 2", None, None)
# finale
F = Match("Finale", None, None)
```

- La première demi-finale D1 verra s'affronter les joueuses victorieuses des quarts de finales Q1 et Q2.

- La seconde demi-finale D2 verra s'affronter les joueuses victorieuses des quarts de finales Q3 et Q4.
 - La finale F verra s'affronter les joueuses victorieuses des demi-finales D1 et D2.
8. Instancier la variable `tournoi` de la classe `Arbre` représentant ce tournoi depuis les quarts de finale en passant par les demi-finales jusqu'à la finale.

Dès qu'un match est joué, et donc dès que la gagnante est connue, on souhaite mettre à jour le match du tour suivant. Par exemple le premier quart de finale Q1 s'est terminé sur le score de 6-4, 7-5, en faveur de la joueuse `gauff`. Elle participera donc en tant que `joueuse1` au premier match des demi-finales D1.

```
Q1.resultat_match([(6, 4), (7, 5)])
```

9. Compléter l'instruction suivante afin de mettre à jour dans `tournoi` la première joueuse de la première demi-finale comme étant `gauff`.

```
tournoi. ... = gauff
```

Cette façon de mettre à jour le tournoi n'est pas satisfaisante, on souhaite automatiser cette tâche à l'aide d'une méthode `mise_a_jour` de la classe `Arbre` qui parcourt l'arbre et dès que l'on rencontre un match dont la gagnante est connue, on la fait passer au tour suivant si ce n'est pas déjà fait. Cette méthode doit être récursive.

10. Rappeler ce qu'est un programme récursif.
11. Recopier et compléter les lignes 6, 7, 13, 14 et 16 du code ci-dessous de la méthode `mise_a_jour` permettant de compléter automatiquement les matchs du tournoi en les mettant à jour à partir des résultats des matchs des tours précédents.

```

1      def mise_a_jour(self):
2          """Met à jour les matchs de l'arbre"""
3          if self.racine.joueuse1 is None:
4              if self.gauche is not None:
5                  # mise à jour si gagnante à gauche
6                  if ...
7                      ... = self.gauche.racine.gagnante
8              else:
9                  self.gauche.mise_a_jour()
10         if self.racine.joueuse2 is None:
11             if self.droit is not None:
12                 # mise à jour si gagnante à droite
13                 if ...
14                     ... = self.droit.racine.gagnante
15             else:
16                 ...

```